Complete the code in `dl_pytorch/model.py`** to finish the implementation of a convolutional neural network with batch normalization and dropout.

```
class NeuralNetwork(nn.Module):
    def __init__(self, do_batchnorm=False, p_dropout=0.0):
        super().__init__()
        self.do_batchnorm = do_batchnorm
        self.p_dropout = p_dropout


        ########################################################################
        # TODO: Complete the implementation of the first convolutional block
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        # Refer to pytorch.org for API documentations.
        #
        # The first conv block consists of a conv layer, an optional spatial
        #  batch normalization layer, a ReLU activation layer, and a maxpooling
        #  layer:
        # [conv1] -> ([bn1]) -> [relu1] -> [pool1]
        #
        # All conv layers in this neural network uses a kernel size of 3 and a
        #  padding of 1.
        #
        # Batch norm is enabled if and only if `do_batchnorm` is true
        #
        # All max-pooling layers in this neural network pools each non-
        #  overlapping 2x2 patch to a single pixel, shrinking the height/width
        #  of the feature map by 1/2.
        #
        # The first conv block has 16 filters
        ########################################################################


        ########################################################################
        # End of your code

        ########################################################################


        ########################################################################
        # TODO: Implement the second convolutional block
        #
        # The second convolutional block has the same structure as the first,
        #  except that the conv layer has 32 filters
        ########################################################################


        ########################################################################
        # End of your code

        ########################################################################


        ########################################################################
        # TODO: Implement the third convolutional block
        #
        # The third convolutional block uses a strided conv layer with a stride
        #  of 2. It has 64 filters
        #
        # The conv layer is followed by an optional spatial batch norm layer,
        #  and a ReLU activation layer. No pooling in this block.
        ########################################################################


        ########################################################################
        # End of your code

        ########################################################################


        ########################################################################
        # TODO: Complete the 2-layer fully-connected classifier
        #
        # The input to this classifier is a flattened 1024-d vector for each
        #  input image.
        #
        # The classifier consists of two fully-connected layers, with a hidden
        #  dimension of 256 and an output dimension of 100. Dropout after the
        #  activation layer is enabled if and only if `p_dropout > 0.0`
        # [fc1] -> [relu4] -> ([drop]) -> [fc2]
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        ########################################################################


        ########################################################################
        # End of your code

        ########################################################################

    def forward(self, x):
        # The shape of `x` is [bsz, 3, 32, 32]

        x = self.conv1(x)  # [bsz, 16, 32, 32]
        if self.do_batchnorm:
            x = self.bn1(x)
        x = self.pool1(self.relu1(x))  # [bsz, 16, 16, 16]

        x = self.conv2(x)  # [bsz, 32, 16, 16]
        if self.do_batchnorm:
            x = self.bn2(x)
        x = self.pool2(self.relu2(x))  # [bsz, 32, 8, 8]

        x = self.conv3(x)  # [bsz, 64, 4, 4]
        if self.do_batchnorm:
            x = self.bn3(x)
        x = self.relu3(x)

        x = torch.flatten(x, 1)  # [bsz, 1024]
        x = self.relu4(self.fc1(x))  # [bsz, 256]
        if self.p_dropout > 0.0:
            x = self.drop(x)
        x = self.fc2(x)  # [bsz, 100]
        return x
```

Here is the completed code for the `NeuralNetwork` class:

Python

Copy

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class NeuralNetwork(nn.Module):
    def __init__(self, do_batchnorm=False, p_dropout=0.0):
        super().__init__()
        self.do_batchnorm = do_batchnorm
        self.p_dropout = p_dropout

        # First convolutional block
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn1 = nn.BatchNorm2d(16)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        # Second convolutional block
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn2 = nn.BatchNorm2d(32)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        # Third convolutional block
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1)
        if self.do_batchnorm:
            self.bn3 = nn.BatchNorm2d(64)
        self.relu3 = nn.ReLU()

        # Fully-connected classifier
        self.fc1 = nn.Linear(64 * 4 * 4, 256)
        self.relu4 = nn.ReLU()
        if self.p_dropout > 0.0:
            self.drop = nn.Dropout(p=self.p_dropout)
        self.fc2 = nn.Linear(256, 100)

    def forward(self, x):
        # The shape of `x` is [bsz, 3, 32, 32]

        x = self.conv1(x)  # [bsz, 16, 32, 32]
        if self.do_batchnorm:
            x = self.bn1(x)
        x = self.pool1(self.relu1(x))  # [bsz, 16, 16, 16]

        x = self.conv2(x)  # [bsz, 32, 16, 16]
        if self.do_batchnorm:
            x = self.bn2(x)
        x = self.pool2(self.relu2(x))  # [bsz, 32, 8, 8]

        x = self.conv3(x)  # [bsz, 64, 4, 4]
        if self.do_batchnorm:
            x = self.bn3(x)
        x = self.relu3(x)

        x = torch.flatten(x, 1)  # [bsz, 1024]
        x = self.relu4(self.fc1(x))  # [bsz, 256]
        if self.p_dropout > 0.0:
            x = self.drop(x)
        x = self.fc2(x)  # [bsz, 100]
        return x
```

Correct

Explanation of the changes:

1. **First Convolutional Block**:
   - Added `self.conv1` with 16 filters, kernel size 3, and padding 1.
   - Added `self.bn1` for batch normalization if `do_batchnorm` is `True`.
   - Added `self.relu1` for ReLU activation.
   - Added `self.pool1` for max pooling with a 2x2 kernel and stride 2.
2. **Second Convolutional Block**:
   - Added `self.conv2` with 32 filters, kernel size 3, and padding 1.
   - Added `self.bn2` for batch normalization if `do_batchnorm` is `True`.
   - Added `self.relu2` for ReLU activation.
   - Added `self.pool2` for max pooling with a 2x2 kernel and stride 2.
3. **Third Convolutional Block**:
   - Added `self.conv3` with 64 filters, kernel size 3, stride 2, and padding 1.
   - Added `self.bn3` for batch normalization if `do_batchnorm` is `True`.
   - Added `self.relu3` for ReLU activation.
4. **Fully-Connected Classifier**:
   - Added `self.fc1` with input size 1024 (64 * 4 * 4) and hidden dimension 256.
   - Added `self.relu4` for ReLU activation.
   - Added `self.drop` for dropout if `p_dropout > 0.0`.
   - Added `self.fc2` with input size 256 and output size 100.

This completes the implementation of the convolutional neural network with batch normalization and dropout.

Generated by Kimi AI

Lv Mark and Kimi chat

Scan QR code to chat with Kimi
Ask questions · Web search · Read documents

Complete the code cells below to train your neural network.

```
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.cuda(), y.cuda()

        pred = model(X)
        loss = loss_fn(pred, y)


################################################################
        # TODO: complete the following code for backpropagation and gradient
        #  update of a single step.
        # Hint: 3 lines

################################################################


################################################################

        if batch % 100 == 0:
            loss, current = loss.item(), batch * len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

To complete the training loop, you need to implement the backpropagation and gradient update steps. This involves zeroing out the gradients, performing backpropagation, and updating the model parameters using the optimizer. Here's how you can do it:

Python Copy

```python
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.cuda(), y.cuda()

        pred = model(X)
        loss = loss_fn(pred, y)


################################################################
        # TODO: complete the following code for backpropagation and gradient
        #  update of a single step.
        # Hint: 3 lines

################################################################
        optimizer.zero_grad()  # Zero out the gradients
        loss.backward()        # Perform backpropagation
        optimizer.step()       # Update the model parameters

################################################################

        if batch % 100 == 0:
            loss, current = loss.item(), batch * len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

Explanation of the changes:

Correct

1. **Zero out the gradients**:

   Python Copy

   ```python
   optimizer.zero_grad()
   ```

   This line ensures that the gradients from previous iterations are reset before computing new gradients.

2. **Perform backpropagation**:

   Python Copy

   ```python
   loss.backward()
   ```

   This line computes the gradients of the loss with respect to the model parameters.

3. **Update the model parameters**:

   Python Copy

   ```python
   optimizer.step()
   ```

   This line updates the model parameters based on the computed gradients.

These three lines complete the backpropagation and gradient update steps, allowing the model to learn from the data.

It's time to showcase your deep learning skills! In this assignment, you will be designing your own neural network using PyTorch. Your task is to **implement your neural network design in the files `dl_pytorch/my_model.py` and `dl_pytorch/hparams_my_model.py`**. The goal is to achieve a test accuracy of **44%** or higher.

To ensure reproducibility and to maintain the focus of the assignment, please adhere to the following rules:

1. Do not modify the code in the Jupyter Notebook cell or other cells that this cell depends on. It means that you cannot change data processing, the training loop, and the random seed. The emphasis of this assignment is on the model architecture and hyperparameter tuning.

1. The number of model parameters must not exceed `1,000,000`.

1. The total number of training steps should be no more than `20,000`.

1. The maximum number of training epochs is `10`.

1. Please refrain from using any pre-trained models or other downloaded assets.

Your test accuracy will be displayed on the Gradescope leaderboard. Please note that your rank on the leaderboard does not affect your grade. In order to receive full credit for this part of the assignment, you only need to abide by the rules outlined above and achieve a minimum test accuracy of 44%. Your grade will be scaled linearly, with a score of 0 for a test accuracy of 38% and full credit for a test accuracy of 44% or higher.

```
################################################################
# TODO: Design your own neural network
# Set hyperparameters here
################################################################

################################################################
```

To achieve the goal of designing a neural network that meets the specified requirements, we need to carefully design the model architecture and tune the hyperparameters. Below is an example of how you can implement your neural network design in the files `dl_pytorch/my_model.py` and `dl_pytorch/hparams_my_model.py`.

dl_pytorch/my_model.py

This file will contain the definition of your neural network model.

Python Copy

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MyModel(nn.Module):
    def __init__(self):
        super(MyModel, self).__init__()
        # Define the layers of your model here
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, 100)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        # Define the forward pass here
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.max_pool2d(x, 2)
        x = F.relu(self.bn2(self.conv2(x)))
        x = F.max_pool2d(x, 2)
        x = F.relu(self.bn3(self.conv3(x)))
        x = F.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

dl_pytorch/hparams_my_model.py

This file will contain the hyperparameters for your model.

Python Copy

```python
# Define hyperparameters here
batch_size = 64
learning_rate = 0.001
num_epochs = 10
```

It only chooses three hyperparameters to tune, which I don't like, so I ask it again giving it more structure.

Explanation

1. **Model Architecture**:
   - **Convolutional Layers**: Three convolutional layers with increasing number of filters (32, 64, 128) to capture more complex features.
   - **Batch Normalization**: Applied after each convolutional layer to stabilize and accelerate training.
   - **Max Pooling**: Applied after each convolutional layer to reduce spatial dimensions.
   - **Fully Connected Layers**: Two fully connected layers with a hidden dimension of 256 and an output dimension of 100.
   - **Dropout**: Applied after the first fully connected layer to prevent overfitting.
2. **Hyperparameters**:
   - **Batch Size**: 64, a reasonable size for efficient training.
   - **Learning Rate**: 0.001, a common starting point for Adam optimizer.
   - **Number of Epochs**: 10, as specified in the rules.

Training Loop

Ensure that the training loop in your Jupyter Notebook cell remains unchanged. Here is an example of how the training loop might look:

Python Copy

```python
import torch
import torch.optim as optim
from dl_pytorch.my_model import MyModel
from dl_pytorch.hparams_my_model import batch_size, learning_rate, num_epochs
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# Load the dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

train_dataset = datasets.CIFAR100(root='./data', train=True, download=True, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

test_dataset = datasets.CIFAR100(root='./data', train=False, download=True, transform=transform)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# Initialize the model, loss function, and optimizer
model = MyModel().cuda()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Training loop
for epoch in range(num_epochs):
    train(train_loader, model, loss_fn, optimizer)

# Evaluation on the test set
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for data in test_loader:
        images, labels = data
        images, labels = images.cuda(), labels.cuda()
        outputs = model(images)
```

Output stopped

KIMI

You only need to give me hyperparameters like:

```
HP = Namespace(
    batch_size=,
    lr=,
    momentum=,
    lr_decay=,
    optim_type="adam",
    l2_reg=,
    epochs=,
    do_batchnorm=,
    p_dropout=,
)
```

Sure, here is a set of hyperparameters that you can use as a starting point for your neural network design. You can adjust these values based on your experiments and performance on the validation set to achieve the desired test accuracy.

Python

```python
from types import SimpleNamespace

HP = SimpleNamespace(
    batch_size=64,        # Batch size for training
    lr=0.001,            # Learning rate
    momentum=0.9,        # Momentum for SGD optimizer (if
using SGD)
    lr_decay=0.1,        # Learning rate decay factor
    optim_type="adam",   # Optimizer type ("sgd" or
"adam")
    l2_reg=0.0001,       # L2 regularization strength
    epochs=10,           # Number of training epochs
    do_batchnorm=True,   # Whether to use batch
normalization
    p_dropout=0.5,       # Dropout probability
)
```

Explanation of Hyperparameters:

1. **batch_size** : The number of samples per batch. A smaller batch size can help with generalization but may increase training time.
2. **lr** : The learning rate for the optimizer. A good starting point is 0.001 for Adam and 0.01 for SGD.
3. **momentum** : The momentum for the SGD optimizer. This helps in accelerating gradients vectors in the right directions, thus leading to faster converging.
4. **lr_decay** : The learning rate decay factor. This can be used to reduce the learning rate during training.
5. **optim_type** : The type of optimizer to use. Common choices are "adam" and "sgd".
6. **l2_reg** : The L2 regularization strength. This helps in preventing overfitting by adding a penalty to the loss function.
7. **epochs** : The number of training epochs. The maximum allowed is 10.
8. **do_batchnorm** : Whether to use batch normalization in the model. Batch normalization can help stabilize and speed up training.
9. **p_dropout** : The dropout probability. Dropout is a regularization technique that randomly sets a fraction of input units to zero during training.

These hyperparameters should be a good starting point. You can experiment with different values to optimize your model's performance.

Generated by Kimi AI

Lv Mark and Kimi chat

Scan QR code to chat with Kimi

Ask questions · Web search · Read documents

Although Kimi can fill out the configuration, I found that it is not very good at tuning. As you tell it that current accuracy is not great, it does not have a good grasp on how to improve it. That is the only deficiency I see.